SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.	5
2	A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.	5
3	A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.	5

4	A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.	5
5	A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.	5
6	A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.	5
7	An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.	5
8	A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.	5

9	A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.	5
10	A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.	5
11	A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.	5
12	An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.	5
13	A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.	5
14	A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.	5

15	A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.	5

Code of Conduct:

I Oviya M / 192321117 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ***oviya m***

SIMATS ENGINEERING

CO2_AT1 Scenario-Based Assessment - Answers

COURSE NAME: Software Testing **COURSE CODE:** CSA37

Name: Oviya M **Register No:** 192321117

COURSE OUTCOME COVERED

CO2: Design effective test cases using black-box, white-box testing techniques and to control methods flow.

1. Introduction and Approach

This document answers the CO2_AT1 Scenario-Based Assessment (CSA37 Software Testing). Each scenario is solved in the order the rubric rewards: (1) understanding the scenario and its context, (2) applying the correct black-box or white-box concepts, (3) analysing alternatives and deciding, (4) designing the solution (test cases, CFG, tables, strategy) and (5) justifying it clearly.

2. Technique Selection Guide

Situation	Technique	Typical output
Numeric limits and ranges	ECP + BVA (robust BVA for invalid edges)	Classes and boundary tables
Several inputs interacting	Decision table, cause-effect graph, pairwise	Rules (2^n) and reduced test set
Workflows with states	State transition testing	State table, 0/1-switch tests
Nested logic and loops	CFG, basis path, loop testing	V(G) and independent paths
Compound conditions	Branch, condition, MC/DC	n+1 test sets
Security / experience risks	Error guessing + path analysis	Attack-to-path map
Mixed risk (financial, safety)	Hybrid black-box + white-box	Layered strategy with exit criteria

3. Formulas and Definitions Used

Cyclomatic complexity: V(G) = E - N + 2 = P + 1. Number of decision-table rules: 2^n for n binary conditions. MC/DC tests: n + 1 for n conditions. Branch coverage = decision outcomes executed / total outcomes. Robust BVA points for range [min,max]: min-1, min, min+1, nominal, max-1, max, max+1.

4. Common Test Adequacy Criteria

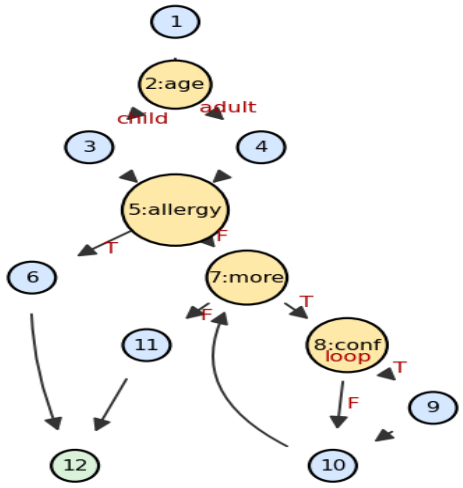
Criterion	Meaning
Statement coverage	every statement executed
Branch coverage	every true/false outcome executed
Condition coverage	every atomic condition true and false
MC/DC	each condition independently affects outcome
Path / basis path	all (or all independent) paths executed
Partition / boundary coverage	every class and boundary has a test

<table>
<tr><th>Sl. No</th><th>Answers</th><th>Bloom's Level</th></tr>
<tr><td>1</td><td>
Scenario analysis: Failures occur exactly at the minimum and maximum transfer limits (assume Rs 100 to 50,000 per transfer). This is the typical <= vs < boundary defect. Random testing rarely produces the exact limit values, and nothing in the current design checks balance, daily limit or format together.
Evaluation of current design: Random data = low yield, no coverage measure, not repeatable, no expected results. It cannot show that limits were ever tested.
Black-box redesign: ECP classes: amount < 100 (invalid), 100-50,000 (valid), > 50,000 (invalid), amount > balance (invalid), non-numeric / zero / negative / >2 decimals (invalid).
<table>
<tr><th>Technique</th><th>Test values</th><th>Expected</th></tr>
<tr><td>BVA at minimum</td><td>99, 100, 101</td><td>Reject, Accept, Accept</td></tr>
<tr><td>BVA at maximum</td><td>49,999, 50,000, 50,001</td><td>Accept, Accept, Reject</td></tr>
<tr><td>Balance boundary</td><td>amount = balance, balance + 1</td><td>Accept, Reject (insufficient funds)</td></tr>
<tr><td>Invalid format</td><td>0, -100, abc, 100.555, blank</td><td>Reject with proper message</td></tr>
</table>
Random testing: retain or replace?: Replace it as the main technique; retain only as a small supplementary/fuzz run for unexpected inputs (seeded so it is repeatable).
Optimised approach: ECP + BVA for each field -> decision table for (limit OK, balance OK, daily limit OK, account active) -> state transition for the transfer life-cycle (Initiated, Authorised, Completed, Failed) -> error guessing and random as extras -> automated regression and a requirement traceability matrix.
Justification: Every limit and rule is tested at its exact edge with an expected result, giving maximum defect detection with a minimal, repeatable test set.
Detailed explanation: A transfer function usually contains the checks: amount format, minimum, maximum, balance, daily total and account status. A fault in any comparison (for example amount > max coded as amount >= max) is only visible at the exact limit value. Because the team used random amounts, such a fault produced failures only occasionally, which matches the symptom reported.
<table>
<tr><th>Test</th><th>Amount</th><th>Balance</th><th>Expected</th></tr>
<tr><td>1</td><td>100</td><td>10,000</td><td>Success (min limit)</td></tr>
<tr><td>2</td><td>99</td><td>10,000</td><td>Rejected: below minimum</td></tr>
<tr><td>3</td><td>50,000</td><td>60,000</td><td>Success (max limit)</td></tr>
<tr><td>4</td><td>50,001</td><td>60,000</td><td>Rejected: above maximum</td></tr>
<tr><td>5</td><td>500</td><td>499</td><td>Rejected: insufficient funds</td></tr>
<tr><td>6</td><td>500</td><td>500</td><td>Success, balance 0</td></tr>
<tr><td>7</td><td>0 / -1</td><td>10,000</td><td>Rejected</td></tr>
</table>
Decision table: Conditions: amount in range, balance sufficient, daily limit not exceeded, account active. Rule 1 (all yes) executes the transfer; each other rule has one failing condition and a specific message. 16 combinations reduce to 5 rules by merging don't-cares.
Comparison: random vs structured: Random: low yield, no coverage evidence, not repeatable. Structured: derived from rules, traceable, repeatable, and gives measurable coverage; therefore random is replaced as the primary technique.
</td><td>5</td></tr>
</table>

2

Evaluation and gaps: Defects reached production because tests followed the normal flow only. Likely achieved: partial statement coverage; branch coverage low (alerts, error paths, loops untested); path coverage not measured. No coverage criterion or exit criteria were defined.

Conceptual CFG (dosage module): 1 read age, weight, allergy | 2 if age<12 | 3 child dose | 4 adult dose | 5 if allergy | 6 block | 7 while more drugs | 8 if interaction | 9 warn | 10 next drug | 11 output | 12 exit.



Cyclomatic complexity: N=12, E=15, **V(G) = 15 - 12 + 2 = 5** (predicates 2, 5, 7, 8).

Path	Route	Test data
P1	1-2-3-5-6-12	child, allergy = yes
P2	1-2-4-5-7-11-12	adult, no drugs (loop skipped)
P3	1-2-3-5-7-8-10-7-11-12	child, 1 drug, no conflict
P4	1-2-4-5-7-8-9-10-7-11-12	adult, drug conflict
P5	1-2-4-5-6-12	adult, allergy = yes

Missing paths / improved strategy: Missing: allergy block, interaction warning, zero and several drugs. Plan: basis path set (P1-P5) + loop tests (0, 1, 2, max drugs) + MC/DC for allergy and dose conditions + BVA on age 11/12 and weight. Exit criteria: 100% statement, 100% branch, all basis paths.

Justification: Safety-related branches (allergy, interaction) are now executed with known expected outcomes, which reduces clinical risk and raises reliability.

Detailed explanation: Defects found after deployment are strong evidence that some branches were never executed. In healthcare modules these are usually error and safety paths (allergy, interaction, missing data) which do not occur in the normal workflow. The testing approach used was functional demonstration, so only the main path was ever executed.

Criterion	Estimated achieved	Target	Gap
Statement	about 70%	100%	dead/error code
Branch	about 50%	100%	allergy, conflict, loop skip
Path	not measured	all basis paths	P1-P5
MC/DC	none	for safety conditions	allergy, dose limit

Risk reduction: Executing allergy and interaction branches with expected results prevents wrong-dose and missing-alert incidents; coverage reports show reviewers exactly which code is still unprotected.

5

3

Scenario and limitation: Price, quantity and discount were validated one field at a time, so errors that appear only for combinations (e.g. 100% discount with large quantity, price at maximum with quantity at stock limit) were missed.

ECP and BVA per field

- **Price:** <= 0 (invalid), 0.01 - max (valid), > max, non-numeric. BVA: 0, 0.01, max, max+0.01.
- **Quantity:** < 1, 1 - stock (valid), > stock, non-integer. BVA: 0, 1, stock, stock+1.
- **Discount %:** < 0, 0 - 100 (valid), > 100. BVA: -1, 0, 100, 101.

Decision table for combinations: Exhaustive invalid/valid cross-product 4 x 4 x 3 is reduced by a decision table:

Rule	Price valid	Qty valid	Disc valid	Result
R1	Y	Y	Y	Total = price x qty x (1 - disc)
R2	N	Y	Y	Price error
R3	Y	N	Y	Quantity error
R4	Y	Y	N	Discount error
R5	N	N	Y/N	Multiple errors listed
R6	Y	Y	Y (disc 0 / 100)	Total = full price / 0

Pairwise (all-pairs) on valid values: 3 parameters x 3 values (min, nominal, max) = 27 combinations reduced to 9 tests that still cover every pair.

Justification: Single-field techniques give correct fields; the decision table and pairwise tests catch interaction defects with far fewer tests, so completeness and defect detection both improve.

Detailed explanation: A retail order total is computed from price, quantity and discount together: total = price x quantity x (1 - discount/100). Single-field tests prove each field is accepted or rejected, but cannot show that the formula behaves correctly when, for example, price is at its maximum and quantity at stock limit (overflow) or discount is 100% (total 0).

Test	Price	Qty	Discount	Expected
1	0.01	1	0	0.01
2	max	1	0	max
3	50	stock	0	50 x stock
4	50	1	100	0.00
5	50	1	101	Discount error
6	0	1	0	Price error
7	50	0	0	Quantity error
8	max	stock	50	No overflow, correct total

Completeness argument: The approach covers all field classes (ECP), all edges (BVA), all validity combinations (decision table) and all valid value pairs (pairwise); anything beyond that is repeated coverage.

5

4	**Gaps in current approach:** Partial path coverage: CFG omitted else edges, loop back edges and compound decisions, so independent paths were not identified and tests followed the happy path. **Impact:** Each untested independent path may hide faults (wrong branch, loop exit, boundary). With loops the path count is unbounded, so exhaustive path testing is impossible; basis paths must be used. **Complete strategy** • Draw full CFG (one node per decision, explicit else, loop back edge, single exit). • Compute V(G) = P + 1 (e.g. 5 predicates -> 6 independent paths). • Derive basis path set and test data; mark infeasible paths with reason. • Add loop tests (0, 1, 2, max-1, max, max+1 iterations) and MC/DC for compound conditions. • Measure with a coverage tool; target 100% statement, 100% branch, all basis paths.	5

Criterion	Expected outcome
Statement 100%	No dead/unexecuted code
Branch 100%	Every true/false and loop exit executed
Basis paths	All V(G) independent paths executed
MC/DC	Each condition proven to affect decision

Justification: Basis paths guarantee every edge is exercised, so critical paths and loop errors are tested in a minimal, countable set.

Detailed explanation: Independent paths are paths that introduce at least one new edge. If the team found only part of them, the graph itself was probably incomplete (missing else edges, loop back edge or compound condition expansion). Fixing the graph is the first step; the path list follows from V(G).

Step	Activity	Output
1	Draw complete CFG	N, E, predicates
2	Compute V(G)	number of paths
3	Select baseline path	P1
4	Flip one decision at a time	P2...Pn
5	Design data per path	test cases with expected output
6	Check feasibility	infeasible paths documented

Expected outcomes: For a module with 5 predicates: V(G) = 6; 6 tests give 100% branch coverage; loop tests add boundary iterations; MC/DC adds condition-level proof.

5	**Analysis of integration:** Techniques ran in silos: black-box tests lacked boundary and combination design; white-box used statement coverage only; no link between requirements, tests and coverage; metrics counted tests, not criteria. **Evaluation:** Adequacy criteria were not defined, so coverage numbers could not show whether partitions, boundaries, branches or paths were covered. **Hybrid strategy** • **Step 1 (black-box):** ECP, BVA, decision table, state testing from the requirements. • **Step 2:** run them and measure code coverage. • **Step 3 (white-box):** build CFG for uncovered code; add branch, basis-path, MC/DC tests. • **Step 4:** risk-based priority for payment, interest and balance logic. • **Step 5:** map every production defect to a missing test and add it to regression. **Exit criteria:** 100% partitions and boundaries; 100% branch (critical modules: MC/DC and basis paths); traceability complete. **Justification:** Black-box finds missing/incorrect behaviour, white-box finds untested code; together they close each other's gaps, and measurable criteria prevent defects from leaking into production. **Detailed explanation:** In the existing project each team applied its own technique independently. Functional testers used examples without boundaries; developers measured statement coverage; nobody compared the two. The result was duplicated tests in the normal flow and untested boundary and error branches. **Defect prevention:** Because every production defect is traced to the layer that should have caught it, the same defect class cannot recur.	5

Layer	Technique	Evidence
Specification	ECP, BVA, decision table	partition and rule coverage
Structure	CFG, branch, basis path	branch and path coverage
Gap analysis	coverage tool report	uncovered branches list
Risk	financial calculations first	MC/DC on interest, fees
Feedback	defect root cause	new regression test

6	**Failure analysis:** Employees at exactly 5, 10 and 15 years get wrong payouts: classic boundary defect (> used instead of >=), so the boundary value falls into the lower band. Tests used only mid-band values. **ECP:** Service years: <0 invalid; 0-4; 5-9; 10-14; 15+; non-numeric invalid. Rating: <1 and >5 invalid; 1-5 valid (assumed scale). **Combination:** Representative band x rating tests (4 x 5 = 20 valid) with boundary ratings at every boundary year. **Why better than exploratory testing:** The bonus rules are deterministic and boundary-sensitive; BVA/ECP target exactly the >= / > fault, are repeatable, auditable and automatable. Exploratory testing depends on tester intuition, gives no coverage guarantee and cannot prove financial correctness. **Detailed explanation:** Suppose the specification says bonus band 2 starts at 5 years (years >= 5). The code uses years > 5. Employees with exactly 5 years receive the band 1 payout: this is exactly the reported failure at 5, 10 and 15 years. BVA tests 4, 5, 6 around each band edge and immediately exposes this error. **Exploratory vs BVA:** Exploratory testing is valuable for discovering unknown issues, but financial logic has explicit numeric rules where exact values matter; BVA gives guaranteed coverage of each rule edge.	5

BVA input (years)	Expected band	Purpose
-1, 0, 1	Invalid, band 1, band 1	lower limit
4, 5, 6	band 1, band 2, band 2	5-year boundary
9, 10, 11	band 2, band 3, band 3	10-year boundary
14, 15, 16	band 3, band 4, band 4	15-year boundary
ratings 0, 1, 5, 6	Reject, Accept, Accept, Reject	rating limits

Years	Rating	Expected bonus band
4	3	Band 1
5	3	Band 2
10	3	Band 3
15	3	Band 4
15	5	Band 4, top rating
15	1	Band 4, lowest rating
0	3	Band 1

7

Assumed logic: alarm if (S1fail && S2fail) || (Temp > Tcrit); else heater/cooler decisions with Temp > High, Temp < Low, else idle.

CFG: N1 read sensors -> N2 S1fail -> N3 S2fail -> N4 Temp>Tcrit -> N5 alarm; false -> N6 Temp>High -> N7 cooler; N8 Temp<Low -> N9 heater; N10 idle -> N11 exit. Predicate nodes 2, 3, 4, 6, 8 give V(G) = 6.

Gap in branch coverage: Tests covered normal heating/cooling only. The branch where both sensors fail with normal temperature (alarm must trigger) was never executed.

MC/DC for (A && B) || C (A = S1fail, B = S2fail, C = Temp>Tcrit): n + 1 = 4 tests:

Test	A	B	C	Result	Shows independence of
T1	T	T	F	Alarm	A (vs T3), B (vs T2)
T2	T	F	F	No alarm	B
T3	F	T	F	No alarm	A
T4	T	F	T	Alarm	C (vs T2)

Strategy: CFG-based branch tests + the MC/DC set + sensor fault sequences (one fails, both fail, stuck value) + fault injection.

Impact: Each condition is proven to influence the alarm decision; hidden masking faults are removed, which is why MC/DC is required for safety-critical software (DO-178C level A).

Detailed explanation: Safety logic often combines sensor faults and temperature thresholds. A branch-coverage test can execute the alarm branch through the temperature path only, and never through the sensor-failure path. MC/DC requires the sensor conditions to be tested independently, which finds the missing alarm.

Criterion	Tests	Finds the sensor-malfunction defect?
Statement	about 3	No
Branch	about 5	Only if the sequence is chosen
MC/DC	n+1 = 4 for the alarm decision	Yes, A and B shown independent

Safety impact: Independent proof of each condition removes masking, supports certification (DO-178C level A) and gives high confidence the alarm triggers whenever required.

5

8	**Limitation of isolated testing:** Each promo code works alone, but stacking creates combinations (two flat codes, % + flat, tier discount) where the total discount exceeds the price and the Total Amount becomes negative. **Conditions:** C1 multiple codes (Y/N), C2 includes flat promo (Y/N), C3 Gold tier (Y/N), C4 total discount >= price (Y/N). Full table = 16 rules; collapsed with don't-cares to 8 (price 100):	5

Rule	C1	C2	C3	C4	Expected action / test
R1	N	-	N	N	Single promo applied normally
R2	N	-	Y	N	Promo + tier discount
R3	N	Y	-	Y	Flat 150 on 100: total = 0, not negative
R4	Y	N	N	N	Two % codes: applied in order, capped
R5	Y	Y	N	N	% + flat applied correctly
R6	Y	Y	-	Y	Flat+flat 120: clamp to 0 or reject stacking
R7	Y	N	Y	Y	% sum > 100: cap at 100%, total 0
R8	-	-	-	-	Invalid/expired code: error message, price unchanged

Optimised test set: 8 tests instead of 16 (don't-care merging) + 2 boundary tests (total = 0.00 and 0.01) remove redundancy while catching all negative-total defects.

Justification: Combinations, not single inputs, cause the defect, so a decision table finds it systematically.

Detailed explanation: The negative total is a combination fault: each code is valid alone but the sum exceeds the price. Cause-effect graphing helps by listing causes (promo types, tier, price) and effects (total, error). A cap rule (total >= 0) is then tested in every combination where discount >= price.

Test	Price	Promo codes	Tier	Expected total
1	100	20%	Regular	80
2	100	20% + 10%	Gold	sequential, >= 0
3	100	flat 150	Regular	0
4	100	flat 60 + flat 60	Regular	0 (clamped) or reject
5	100	100% + flat 10	Gold	0
6	0.01	flat 5	Regular	0.00

Optimisation: Merging don't-care conditions reduces 16 rules to 8, and boundary total (0.00 and 0.01) adds only 2 tests: minimal redundancy with maximum defect detection.

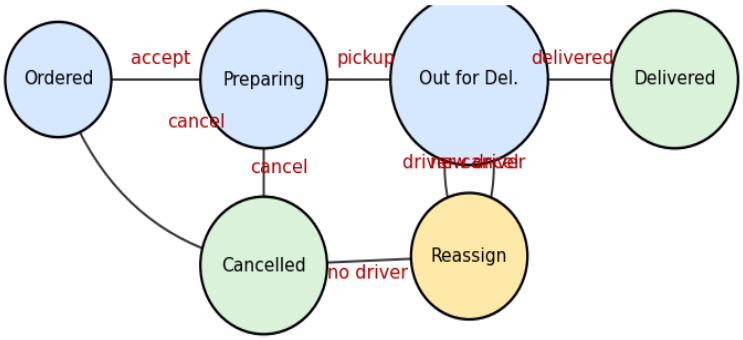
9	**Evaluation:** Statement coverage only shows each line ran once. A loop body can run once and still be infinite in another case (condition variable not updated in a branch/continue); loop exit conditions, zero iterations and boundaries are not checked. **Path testing strategy:** Draw CFG of outer loop (records) and inner loop (fields); predicates: outer loop, inner loop, field-invalid, retry, end-of-file (P = 5). **Cyclomatic complexity: V(G) = P + 1 = 6**: six independent paths (minimum tests): skip outer loop; outer once, inner skipped; inner once; field invalid (path with retry); retry limit reached; EOF exit. **Loop boundary conditions:** For each loop: 0, 1, 2, max-1, max, max+1 iterations; nested: innermost first, outer at minimum, then outer at typical/max; add a timeout/iteration guard test to prove termination. **Justification:** V(G) = 6 is an acceptable risk level (above 10 would suggest refactoring) and gives the exact number of basis paths needed; loop boundary tests expose the infinite loop that statement coverage hid. **Detailed explanation:** Example: while(i<n){ if(rec.bad) continue; i++; }. A test with only good records runs every statement (100% statement coverage) but a bad record skips the increment, creating an infinite loop. Only a path through the continue branch with a bad record reveals it.	5

Path	Description	Test
1	Outer skipped	0 records
2	One good record	1 record
3	Bad record then good	exposes continue path
4	Inner loop 0 fields	record with no fields
5	Retry limit reached	record failing 3 times
6	End of file exit	last record

Cyclomatic complexity interpretation: V(G) 1-10 = simple, 11-20 = moderate, above 20 = high risk. A nested legacy loop with V(G) over 10 should be refactored as well as tested.

10

Evaluation of current strategy: ECP/BVA tested single inputs (address, order id) but not sequences of events; "driver cancels while Out for Delivery" has no defined transition, so the app has no exit from the state.



Current state	Event	Next state	Test type
Ordered	restaurant accepts	Preparing	valid
Preparing	food picked up	Out for Delivery	valid
Out for Delivery	delivered	Delivered	valid
Out for Delivery	driver cancels	Reassigning (missing)	valid (defect)
Reassigning	new driver / timeout	Out for Delivery / Cancelled + refund	valid
Delivered	any cancel event	stays Delivered, error	invalid
Ordered	"delivered" event	rejected	invalid

Test scenarios: 0-switch: all valid transitions; 1-switch: Out for Delivery -> Reassigning -> Out for Delivery -> Delivered; plus all invalid transitions.

Justification: ECP and BVA treat inputs independently; here the same event gives different results depending on state and history. State transition testing exposes unhandled transitions and dead-end states in dynamic workflows.

Detailed explanation: State transition testing uses a state table with every state-event pair. Pairs with no defined behaviour are bugs. The cancel event in Out for Delivery had no entry, which is why the app stayed stuck.

State \ Event	Accept	Pickup	Delivered	Customer cancel	Driver cancel
Ordered	Preparing	-	-	Cancelled	-
Preparing	-	Out for Del.	-	Cancelled	-
Out for Del.	-	-	Delivered	Cancelled + fee	Reassigning
Reassigning	-	Out for Del.	-	Cancelled	(new driver) Out for Del.
Delivered	-	-	-	Error	Error

Why ECP/BVA miss this: They test input values, not event order. The same input (cancel) has different expected results depending on the state, which only a state model captures.

5

<table>
<tr>
<td>11</td>
<td>
Assessment: Black-box tests checked password strength, OTP and IP in valid/invalid combinations only; white-box analysis did not examine the failed-attempt counter, lockout and rate-limit paths, so unlimited attempts remained possible.

Hybrid strategy: (1) Build CFG of authentication: password check -> fail counter -> lock decision -> OTP check (expiry) -> IP check -> grant. (2) Basis-path tests for each path. (3) Use error guessing to create attack data and map each guess to a path:
<table>
<tr><th>Error guess (attack)</th><th>Path targeted</th><th>Expected</th></tr>
<tr><td>Repeated wrong passwords (N+1 attempts)</td><td>fail counter / lock branch</td><td>Account locked, delay</td></tr>
<tr><td>OTP guessed 000000-999999, reused or expired OTP</td><td>OTP expiry / retry branch</td><td>Rejected, limited tries</td></tr>
<tr><td>Spoofed IP header, new location</td><td>IP check branch</td><td>Extra verification</td></tr>
<tr><td>Parallel requests (race), cookie reset to bypass counter</td><td>counter update</td><td>Counter stays server-side</td></tr>
<tr><td>Common passwords, SQL injection strings</td><td>password validation</td><td>Rejected</td></tr>
</table>
Justification: Path analysis guarantees no authentication path is untested; error guessing supplies realistic attack data for those paths. Together they close the brute-force gap and strengthen the security posture.

Detailed explanation: Brute-force vulnerability means the attempt counter or rate limiter does not stop repeated tries. The CFG shows a path from the failed-password node back to the password prompt without passing through the lock decision (counter never checked). Path analysis proves this path exists; error guessing supplies the attack data to exercise it.
<table>
<tr><th>Authentication path</th><th>Test</th></tr>
<tr><td>Correct password + correct OTP + trusted IP</td><td>grant</td></tr>
<tr><td>Wrong password (1st-N-1)</td><td>retry, counter++</td></tr>
<tr><td>Wrong password N-th</td><td>lock account</td></tr>
<tr><td>Correct password, wrong OTP</td><td>retry limit applies</td></tr>
<tr><td>Expired OTP</td><td>reject, new OTP</td></tr>
<tr><td>Unknown IP</td><td>extra verification</td></tr>
</table>
Security posture: Every path is executed, every guessed attack is mapped to a path, so the module can show evidence that brute force, replay and spoofing attempts are rejected.
</td>
<td>5</td>
</tr>
</table>

12	**Evaluation:** Testing used typical dates only; the upgrade on the exact expiry day (a boundary) was not covered. **ECP:** Tier: Free, Basic, Premium. Date vs expiry E: before (E-1), on E, after (E+1, lapsed), invalid/blank date. Course type: Free, Basic, Premium.	5

Test	Current tier	Action date	Expected
T1	Basic	E-1, upgrade to Premium	Premium immediately, expiry extended
T2	Basic	E (exact day)	Premium, no access gap or double charge
T3	Basic	E+1	Treated as lapsed/renewal, Free until paid
T4	Free	any date, upgrade to Basic	Basic access
T5	Premium	E, downgrade to Basic	Rules per policy
T6	Any	blank/invalid date	Error message

Access matrix: 3 tiers x 3 course types = 9 tests (allowed/denied) to confirm correct course access after each transition.

Justification: Classes are disjoint and cover all date relations; boundary dates (E-1, E, E+1) and the matrix cover all transitions, so users move between tiers seamlessly.

Detailed explanation: Expiry-day handling depends on whether the expiry date is inclusive. If the system compares today > expiry the user still has access on the expiry day; if it compares today >= expiry access ends that day. The upgrade at E exposes inconsistent comparisons between the billing and access modules.

Date relation	Class	Test dates
Before expiry	valid active	E-30, E-1
Expiry day	boundary	E
After expiry	lapsed	E+1, E+30
Invalid	error	blank, 30 Feb, far future

Completeness: Date classes x 3 tiers x upgrade/downgrade give 4 x 3 x 2 = 24 combinations; boundary tests (E-1, E, E+1) are selected for each upgrade path.

13

Risk of statement coverage only: switch with no default or missing break (fall-through), hidden false outcomes of if conditions and short-circuit Boolean evaluation can all reach 100% statement coverage without being tested. Wrong dim level or an unknown mode value would reach the hardware.

Example: switch(mode){AUTO: if(lux<50 && !manual) 80; else if(lux<200 || timer) 40; else 0; NIGHT: 10; OFF: 0; (no default)}

Criterion	Tests needed
Branch	AUTO, NIGHT, OFF, undefined mode (default); each if both true/false
Condition	lux<50 T/F, manual T/F, lux<200 T/F, timer T/F
Sample set	(lux 10, manual F)->80; (lux 10, manual T)->40/0; (lux 150, timer F)->40; (lux 300, timer T)->40; (lux 300, timer F)->0; mode=99

Improved strategy: Branch + Condition (or MC/DC) coverage, BVA on lux (0, 49, 50, 199, 200, max), tests for default/invalid mode, sensor failure values.

Justification: Every decision outcome and condition is exercised, so wrong levels and unhandled sensor/mode values are found before integration, reducing hardware-software integration failures.

Detailed explanation: A switch without default and without break can fall through to the next case, executing statements that statement coverage still counts as covered. Branch coverage requires a test per case plus default. Condition coverage requires each Boolean operand to be both true and false.

Test	mode	lux	manual	timer	Expected level
1	AUTO	10	F	-	80
2	AUTO	10	T	F	40 (second branch)
3	AUTO	150	F	F	40
4	AUTO	300	F	T	40
5	AUTO	300	F	F	0
6	NIGHT	-	-	-	10
7	OFF	-	-	-	0
8	99	-	-	-	default/safe

Risk reduction: Unknown values from the hardware are handled by default; thresholds are checked at their edges; so mismatched sensor ranges between hardware and software are found in unit tests.

5

14

Failure analysis: Input validation tests used only normal weights; zero and negative weights were never tested, so the cost formula returned zero/negative shipping charges.

Robust BVA: Assume valid weight 0.1 to 50 kg. Robustness points: min-1, min, min+1, nominal, max-1, max, max+1.

Weight (kg)	Class	Expected
-1, 0	Invalid (<= 0)	Reject, error message
0.1, 0.2	Valid min / min+	Cost calculated
25	Nominal	Cost calculated
49.9, 50	Valid max-1 / max	Cost calculated
50.1	Invalid (> max)	Reject
blank, abc	Invalid format	Reject

Also: Invalid zone and speed values, and combinations with weight at the limits.

Justification: Negative scenarios prove that invalid data is rejected, preventing zero/negative charges, revenue loss, wrong labels and crashes; a robust system must handle misuse and not only correct use.

Detailed explanation: Robust BVA extends normal BVA by adding invalid values just outside the range. For weight in [0.1, 50] this adds -1, 0 and 50.1. Invalid values must be rejected before any cost calculation, otherwise the formula multiplies by zero or a negative number.

Weight	Zone	Speed	Expected
0	A	Standard	Reject
-5	A	Express	Reject
0.1	A	Standard	Min cost
50	C	Express	Max cost
50.1	C	Express	Reject
10	unknown	Standard	Reject zone
10	A	unknown	Reject speed

Importance of negative tests: Real users and integrations send invalid data; the system must fail safely with messages, never with wrong prices or crashes.

5

15

Evaluation of random testing: Random sizes seldom hit prime values (no equal split into parallel chunks, remainder handled wrongly), and random results have no oracle. The inconsistency therefore never appeared.

Basis path testing: Chunking logic: 1 if n<=0 | 2 if n<T (threads) | 3 chunk=n/T, rem=n%T | 4 for each thread | 5 if rem>0 give extra items | 6 merge results. P = 4 -> **V(G) = 5**.

Path	Test data	Expected
P1	n=0	Error/empty result
P2	n < T (e.g. n=3, T=8)	Correct result, idle threads
P3	n divisible by T (n=1024)	Correct, rem=0
P4	n prime (n=1009)	Correct, remainder handled
P5	n = T, n = T+1	Correct

Additional checks: Compare parallel output with sequential reference (tolerance for floating point), run with a race detector, repeat for several thread counts.

Justification: Each independent path has a known expected result, including the prime-size path, so mathematical accuracy is proven and thread/chunk errors that cause instability are removed.

Detailed explanation: Parallel loops split the data into equal chunks. If n is a multiple of the thread count each thread gets the same amount. When n is prime there is always a remainder; code that ignores the remainder loses the last items, giving small inconsistent results that random sizes seldom expose.

n	Threads T	chunk, remainder	Expected
1024	8	128, 0	correct
1009 (prime)	8	126, 1	last item included
7 (prime)	8	0, 7	n < T handled
8	8	1, 0	correct
0	8	-	empty result / error

Accuracy and stability: The parallel result must match the sequential reference within a defined tolerance for every path, and repeated runs must be identical (no race conditions). This proves mathematical accuracy and stable behaviour.

5

5. Consolidated Summary of Scenarios

Q	Domain	Main problem	Technique(s) applied
1	Banking transfer	Limit defects; random testing	ECP, BVA, decision table, state
2	Healthcare	Insufficient coverage	CFG, basis path, branch coverage
3	E-commerce	Input combinations missed	ECP, BVA, decision table, pairwise
4	Nested logic	Partial path coverage	CFG, V(G), loop tests, MC/DC
5	Financial	Techniques not integrated	Hybrid ECP/BVA + path testing
6	HR payroll	Errors at 5, 10, 15 years	BVA, ECP
7	Lab temperature	Alarm branch missed	CFG, MC/DC
8	Travel promo	Negative total	Decision table / cause-effect
9	DB migration	Infinite loops	Basis path, loop boundaries, V(G)
10	Food delivery	Stuck state	State transition testing
11	Authentication	Brute force	Error guessing + path analysis
12	E-learning	Upgrade at expiry	ECP, BVA on dates
13	Smart home	Statement coverage only	Branch, condition coverage
14	Logistics	Zero/negative weight	Robust BVA
15	Medical imaging	Random testing, prime sizes	Basis path testing

6. Overall Conclusion

Across all fifteen scenarios the failures came from weak test design rather than from a lack of testing effort: random data instead of boundaries, single-field checks instead of combinations, statement coverage instead of branch or path coverage, and tests without a state model. The proposed solutions replace each weakness with a technique that matches the type of logic in the module, and add a measurable adequacy criterion so that completeness can be shown. A layered approach (specification-based first, structure-based second, experience-based last) provides the best defect detection for the effort and the lowest production risk.

7. Recommended Test Process

1. Analyse requirements and risks. 2. Select techniques per module type. 3. Derive test cases with expected results. 4. Execute and measure coverage. 5. Add white-box tests for uncovered branches/paths. 6. Review with traceability matrix. 7. Automate regression and feed every production defect back as a new test.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification